Prompt Trees and Random Forests for Time Series Classification

Marco Santi VR504613 and Daniele Pasotto VR504755
Biomedical Decision Support Systems

February, 2026

1 Introduction

Time series classification often requires analyzing sequential data to detect patterns that may occur at variable time intervals. Traditional methods may fail to capture these local temporal features. This project implements **Prompt Trees**, a specialized decision tree architecture formally defined to handle slice-based tests.

A Prompt Tree T classifies a time series x by routing it through nodes containing Reference Slice Tests (RST) or Channel Reference Slice Tests (CRST). Each test evaluates whether the distance between a subsequence of x and a learned reference pattern $\bar{x}$ satisfies a threshold ϵ.

2 Methods

2.1 The Prompt Tree Implementation

The core of the project is the `PromptTree` class (in `tree.py`). The fitting process follows the *Prompt TreeFit* algorithm, which recursively partitions the data using a 5-step optimization procedure.

2.1.1 Tree Functions

The tree uses five key functions defined in `tree_functions.py`:

1. **Promptness Function (f_p)**: Implemented as `fp_default`. It generates candidate time intervals $[b, e]$ for testing. To ensure structural constraints, the start index b is constrained to be either a reuse of an ancestor's start index or the maximum end index seen so far plus one ($max(E) + 1$). The implementation uses sliding windows (e.g., lengths 5, 10, 20) to propose valid intervals. This approach is preferred over an exhaustive search (testing every possible end index $e > b$) for computational efficiency. It ensures the tree searches for patterns at specific temporal scales (short, medium, and long durations) rather than overfitting to arbitrary slice lengths.

2. **Sampling Function (f_s)**: Implemented as `fs_default`. It reduces the search space by randomly sampling K_{refs} reference patterns from the training data. For each reference, it computes distances to all samples and selects candidate thresholds ϵ based on percentiles (25th, 50th, 75th). Using percentiles allows the thresholds to adapt to the distribution of distances for that specific feature. A low percentile (25th) creates a "strict" test that only admits very similar shapes, while a high percentile (75th) creates a "loose" test. Testing multiple thresholds increases the diversity of candidate splits available to the optimizer.

Algorithm 1 PromptTreeFit

Require:

X - Set of time series data
Y - Labels (optional, can be None for unsupervised learning)
Path - Current path in the tree (None for root node creation)
f_p - Promptness function: $2^{\mathbb{N}} \times \mathbb{N} \times \text{Dom}(X) \times \text{Dom}(Y) \to 2^{\mathbb{N} \times \mathbb{I}(\mathbb{N})}$
f_s - Sampling function: $\text{Dom}(X) \times \text{Dom}(Y) \times 2^{\mathbb{N} \times \mathbb{I}(\mathbb{N})} \to 2^{\mathbb{N} \times \mathbb{R}^k \times \mathbb{R} \times \mathbb{I} \times \Delta}$
f_c - Classification function for leaf nodes
Δ - Set of distance functions: $\cup_{k=1}^{\infty} \mathbb{R}^k \to \mathbb{R}$
f_o - Optimization function: $\text{Dom}(X) \times \text{Dom}(Y) \times 2^{\mathbb{N} \times \mathbb{R}^k \times \mathbb{R} \times \Delta} \to \mathbb{N} \times \mathbb{R}^k \times \mathbb{R} \times \mathbb{I} \times \Delta$
f_e - Stopping criterion: $2^{\text{Path}} \times \text{Dom}(X) \times \text{Dom}(Y) \to \{\text{true}, \text{false}\}$

Ensure: A fitted prompt tree node and its subtree

if Path = None **then**
  Path $\leftarrow \emptyset$ ▷ Initialize empty path for root node
  $B \leftarrow \{0\}$ ▷ Root test starts at beginning of time series
else
  $B \leftarrow \{b_i \mid (\overline{x}_i, c_i, b_i, e_i, \delta_i, \varepsilon_i) \in \text{Path}\}$ ▷ Extract beginning indices from path
  $E \leftarrow \{e_i \mid (\overline{x}_i, c_i, b_i, e_i, \delta_i, \varepsilon_i) \in \text{Path}\}$ ▷ Extract ending indices from path
end if
if $f_e(\text{Path}, X, Y) = \text{true}$ **then**
  return LeafNode($f_c(X, Y)$) ▷ Create leaf with classification distribution
end if
CandidateIntervals $\leftarrow f_p(B, \max(E) + 1, X, Y)$ ▷ Get candidate intervals
CandidateTests $\leftarrow f_s(X, Y, \text{CandidateIntervals})$ ▷ Generate candidate tests
$(c, \overline{x}, \varepsilon, [b, e], \delta) \leftarrow f_o(X, Y, \text{CandidateTests})$ ▷ Select optimal test
Node $\leftarrow$ InternalNode$(c, \overline{x}, b, e, \delta, \varepsilon)$ ▷ Create node with selected test
$X_{\text{true}}, Y_{\text{true}} \leftarrow \text{Split}(X, Y, (c, \overline{x}, b, e, \delta, \varepsilon), \text{true})$
$X_{\text{false}}, Y_{\text{false}} \leftarrow \text{Split}(X, Y, (c, \overline{x}, b, e, \delta, \varepsilon), \text{false})$
$\text{Path}_{\text{true}} \leftarrow \text{Path} \cup \{(c, \overline{x}, b, e, \delta, \varepsilon)\}$ ▷ Extend path with current test
Node.left $\leftarrow$ PromptTreeFit$(X_{\text{true}}, Y_{\text{true}}, \text{Path}_{\text{true}}, f_p, f_s, f_c, \Delta, f_o, f_e)$
$\text{Path}_{\text{false}} \leftarrow \text{Path} \cup \{(c, \overline{x}, b, e, \delta, \varepsilon)\}$ ▷ Extend path with current test
Node.right $\leftarrow$ PromptTreeFit$(X_{\text{false}}, Y_{\text{false}}, \text{Path}_{\text{false}}, f_p, f_s, f_c, \Delta, f_o, f_e)$
return Node

Figure 1: The Prompt Tree Fit algorithm

3. **Optimization Function (f_o)**: Implemented as `fo_default`. It iterates through all candidate tests generated by f_s and selects the one that maximizes Information Gain (using Gini impurity reduction). Gini impurity measures the frequency at which a randomly chosen element from the set would be incorrectly labeled if it were randomly labeled according to the distribution of labels in the subset. The gain is calculated as:

$$Gain = Gini_{parent} - (w_L \cdot Gini_{Left} + w_R \cdot Gini_{Right})$$

where w is the fraction of samples in each child node. Maximizing this gain ensures the split effectively separates the classes.

4. **Classification Function (f_c)**: Computes the class distribution at leaf nodes, enabling probabilistic predictions.

5. **Distance Functions (Δ)**: The system uses a set of distance functions to compare time series slices. The default implementation uses Euclidean Distance ($L2$), defined as:

$$\delta(a, b) = \sqrt{\sum (a_i - b_i)^2}$$

This calculates the similarity between the candidate slice $x[c][b : e]$ and the reference $\bar{x}$.

6. **Stopping Criterion (f_e)**: Implemented as `fe_default`. It halts recursion if the maximum depth is reached, the number of samples is below a minimum threshold, or the node is pure. A **pure node** is a node containing samples that all belong to the same class. In this state, the Gini impurity is 0, and no further splitting can improve classification performance.

2.1.2 Handling "Too-Early" Cases

A critical aspect of Prompt Classification is handling time series that are shorter than the required slice interval $[b, e]$. The implementation in `tree.py` explicitly checks:

```
if node.e > x_sample[node.c].shape[-1]:
    return -1  # 'Too early'
```

This adheres to the definition of prompt classification where functions map to $\mathcal{L} \cup \{\text{too-early}\}$.

2.2 Post-Pruning Strategy

To avoid overfitting, a **Reduced Error Pruning (REP)** strategy is implemented. The method `_prune_recursive` in `tree.py` performs a bottom-up traversal using a dedicated validation set.

For every internal node, the algorithm compares:

- **Accuracy as Subtree:** The accuracy on the validation set if the current subtree is kept.
- **Accuracy as Leaf:** The accuracy on the validation set if the subtree is replaced by a single leaf node predicting the majority class.

If the leaf accuracy is greater than or equal to the subtree accuracy, the node is pruned. This creates a simpler model that generalizes better to unseen data.

2.3 Random Forest Architecture

The `RandomForest` class (in `forest.py`) extends the single tree concept to an ensemble.

2.3.1 Bootstrapping and OOB

The implementation supports bootstrapping (if the parameter is given), creating different trees by training on random subsets of the data sampled with replacement. When bootstrapping is used, some samples are left out of the training set for any given tree. These are called **Out-of-Bag (OOB)** samples. The system calculates the prediction accuracy on these OOB samples to provide an unbiased estimate of the generalization error during training, removing the need for an external validation set for this specific metric.

2.3.2 Voting Mechanisms

Three voting mechanisms are implemented for prediction:

- **Majority:** Each tree casts a single vote for its predicted class. The majority among all the trees is the predicted class.
- **Weighted:** This uses "soft voting." Instead of counting binary votes, the *predicted probabilities* for each class are averaged across all trees. This takes into account the confidence of each tree; a tree that is 90% sure of a prediction has more influence than a tree that is only 51% sure.
- **Track-Record:** Trees are weighted based on their individual OOB accuracy. Trees that performed well on their OOB samples during training are assigned a higher weight in the final voting process, effectively giving "smarter" trees more say in the decision.

3 Unsupervised Learning and Clustering

3.1 Isolation Forest Adaptation

The framework supports unsupervised learning by adapting the Prompt Tree logic. When mode is set to `unsupervised`, the optimization function f_o is switched to `fo_unsupervised_random`. Instead of optimizing for class separation, this function selects random valid splits to isolate data points. Anomaly scores are calculated based on the average path length required to isolate a sample, following the standard Isolation Forest logic.

3.2 Distance Metrics for Clustering

To evaluate the unsupervised structure, three distance metrics are implemented in `distances.py`. These metrics define similarity based on how samples traverse the forest.

3.2.1 Breiman Distance

Derived from the proximity matrix P, where P_{ij} is the fraction of trees where sample i and j land in the same leaf.

$$D_{Breiman}(i,j) = \sqrt{1 - P_{ij}}$$

The implementation uses vectorized NumPy broadcasting for efficiency.

3.2.2 Zhu Distance

Based on the depth of separation. For each pair of samples, the implementation finds the Lowest Common Ancestor (LCA) in every tree.

$$D_{Zhu}(i,j) = \frac{1}{T}\sum_{t=1}^{T}(max_depth_t - depth(LCA_{ij}))$$

This metric penalizes samples that diverge early in the tree (near the root).

3.2.3 RatioRF Distance

A ratio-based metric that emphasizes conflict between trees.

$$D_{RatioRF}(i,j) = \frac{\text{Count}_{Apart}(i,j)}{\text{Count}_{Together}(i,j)}$$

The implementation handles division-by-zero (infinite distance) by replacing it with a large finite value ($2 \times max_finite$), ensuring compatibility with clustering algorithms.

3.3 Clustering Procedure

A hierarchical clustering procedure (Average Linkage) is applied using the computed distance matrices. The algorithm iteratively merges clusters until the number of clusters equals the number of true classes in the dataset.

The resulting clusters are evaluated using:

- **Internal Validation:** Average Intra-cluster and Inter-cluster distances.
- **External Validation:** Purity and Entropy (Gini) relative to ground truth labels.
- **Comparison:** Adjusted Rand Index (ARI) to compare the structural agreement between Breiman, Zhu, and RatioRF clusterings.

4 Conformal Prediction

To quantify predictive uncertainty, a Conformal Classifier is implemented.

4.1 Calibration

The training data is split into a *Training Set* and a *Calibration Set* (controlled by `calibration_ratio` parameter). The Random Forest is trained only on the training set.

4.2 Nonconformity Scores

For every sample in the calibration set, a nonconformity score α_i is calculated:

$$\alpha_i = 1 - \hat{P}(y_{true}|x_i)$$

This measures how "surprising" the true label is given the model's prediction. If the model assigns a high probability to the true class, the score is close to 0 (conformity). If the model assigns a low probability to the true class, the score is close to 1 (nonconformity).

4.3 Prediction Sets

For a new test sample, a prediction set is generated containing all labels y such that:

$$1 - \hat{P}(y|x_{new}) \leq \hat{q}_\epsilon$$

where $\hat{q}_\epsilon$ is the $(1 - \epsilon)$ quantile of the calibration scores. This ensures that the true label is contained in the set with probability $1 - \epsilon$ (validity). The **Efficiency** of the predictor is evaluated by the average size of these prediction sets.

5 Experiments and Results

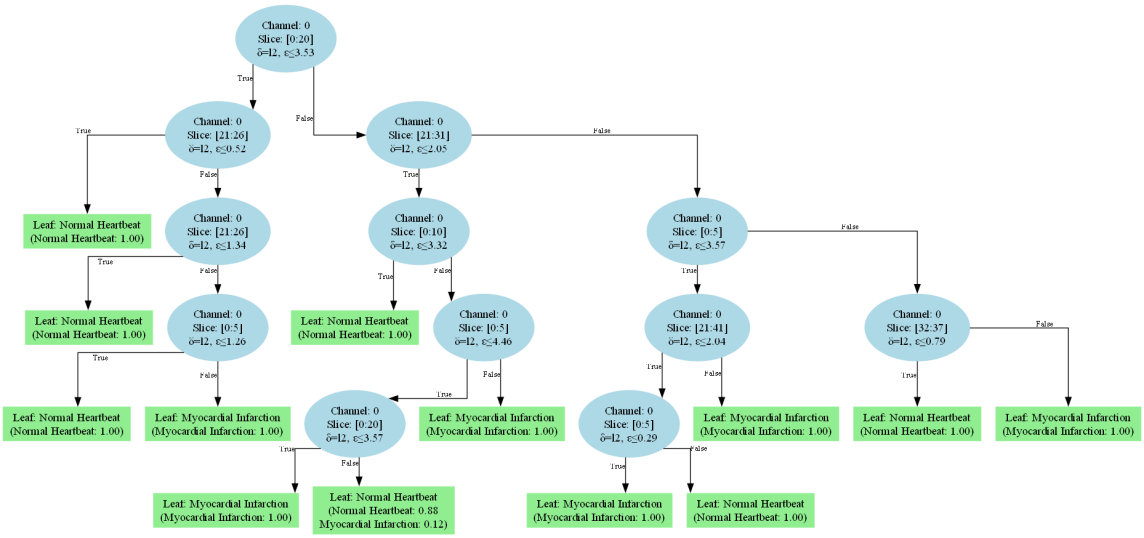
Following the assignment requirements, the evaluation scenarios are tested.

5.1 Univariate + Binary Classification (ECG200)

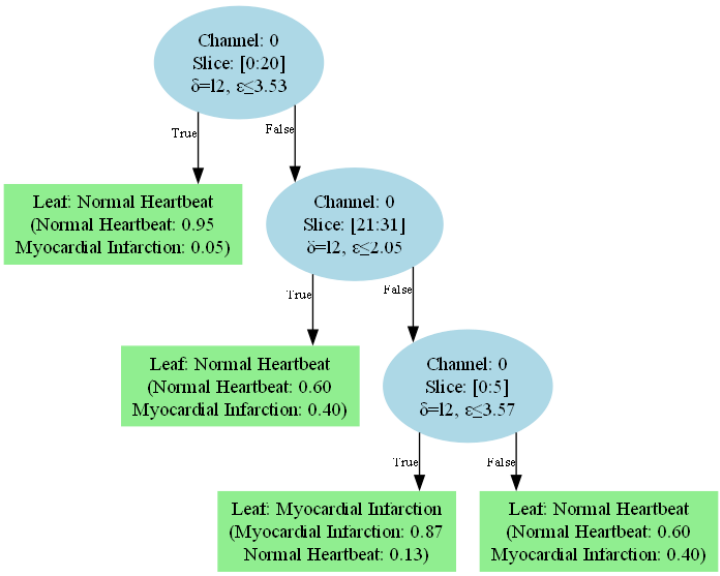
The **ECG200** dataset was selected to evaluate the performances in a univariate binary classification context, specifically distinguishing between *Normal Heartbeat* and *Myocardial Infarction*.

5.1.1 Supervised: Tree Pruning and Generalization

A single `PromptTree` was initially trained, allowing it to fully capture the 100-sample training set. As expected, the unpruned tree exhibited signs of overfitting. To address this, we applied **Reduced Error Pruning (REP)** using a dedicated validation set.
The pruning process successfully condensed the model from 13 leaves to just 4 leaves.
Unlike previous iterations, this specific experiment demonstrated that pruning effectively addressed overfitting. The original tree achieved a test accuracy of 71.00%, while the pruned tree improved performance to **78.00%**. This 7% improvement in test accuracy confirms that reducing the complexity of the Prompt Tree allowed it to focus on more robust features, leading to better generalization on unseen ECG data.



(a) Original Tree (Overfitted)



(b) Pruned Tree (Generalized)

Figure 2: Prompt Tree structure for ECG200. Pruning successfully reduced the complexity of the tree.

5.1.2 Unsupervised: Clustering and Distance Comparison

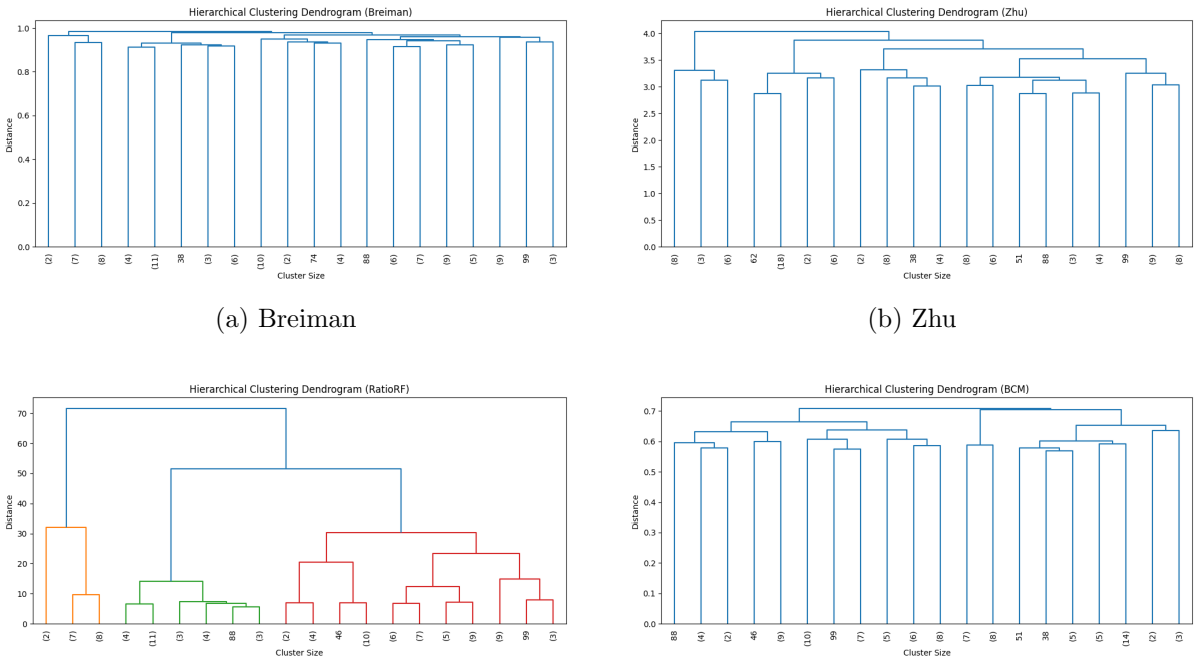
An Isolation Forest consisting of 100 estimators was fitted to the ECG200 data. Then, pairwise distance matrices are generated using the three required metrics: **Breiman**, **Zhu**, and **RatioRF**. These matrices served as input for hierarchical clustering with average linkage.

Metric	ARI	Purity	Entropy	Intra-Dist	Inter-Dist
Breiman	0.1744	0.7400	0.8204	0.9561	0.9846
Zhu	0.1744	0.7400	0.8204	3.5351	4.0369
RatioRF	0.1744	0.7400	0.8204	32.6039	71.6105
BCM	**0.3785**	**0.8100**	**0.5606**	0.6352	0.7084

Table 1: ECG200 Unsupervised Clustering Performance

A notable outcome of this experiment was the convergence of all three baseline distance metrics toward the exact same clustering partition. As indicated by the identical Adjusted Rand Index (0.1744) and Purity (0.7400) scores, the Breiman, Zhu, and leaf-based RatioRF methods grouped the data identically. However, these metrics exhibited profound differences in their geometric interpretation of the feature space. The leaf-based RatioRF yielded the most pronounced separation, characterized by an average inter-cluster distance (71.61) that was more than twice its intra-cluster distance. Conversely, the Breiman distance compressed the data into a remarkably narrow numerical band (0.95 to 0.98), resulting in a mathematically weaker separation despite producing the same final structural hierarchy.

While the leaf-based and common-prefix metrics converged to a suboptimal local structure, the true path-based RatioRF implementation (BCM) demonstrated a marked superiority in uncovering the underlying data distribution. By evaluating tests across diverging paths to capture hidden commonalities, BCM achieved a substantially higher Adjusted Rand Index (0.3785 compared to the 0.1744 baseline) and improved overall cluster Purity to 0.8100. Furthermore, the cluster Entropy dropped to 0.5606, indicating significantly cleaner partitions with fewer mixed labels. This quantitative improvement is visually corroborated by the hierarchical dendrograms (Figure 3): whereas the Breiman, Zhu, and leaf-based RatioRF metrics suffered from severe "chaining"—peeling off small subsets sequentially from a single dense block—BCM produced a highly balanced, structured tree that successfully bifurcated the dataset into two distinct and meaningful signal types.



(a) Breiman

(b) Zhu

(c) RatioRF

(d) BCM

Figure 3: Hierarchical clustering dendrograms for ECG200. The Breiman (a), Zhu (b), and RatioRF (c) metrics exhibit significant chaining, failing to find deep structure. In contrast, BCM distance (d) successfully uncovers a balanced and structured hierarchy, correlating with its vastly superior Adjusted Rand Index.

5.1.3 Conformal Prediction: Uncertainty Quantification

A conformal classifier is implemented to evaluate the reliability of the Random Forest predictions. Setting a significance level $\epsilon = 0.1$, the model was calibrated using a 30% split of the

training data.

Estimators (N)	**Empirical Error Rate**	**Calibration Status**	**Avg Set Size**
20	0.120	Miscalibrated (Error $> \epsilon$)	1.060
100	0.080	**Adequately Calibrated**	1.150

Table 2: Conformal Evaluation for ECG200 ($\epsilon = 0.1$)

With only 20 estimators, the model was slightly miscalibrated (12% error vs. 10% tolerance). However, increasing the forest size to 100 estimators allowed the model to reach a valid calibration state (8% error) with high efficiency, maintaining an average prediction set size of 1.15. This demonstrates that the forest provides well-calibrated confidence intervals when given sufficient capacity.

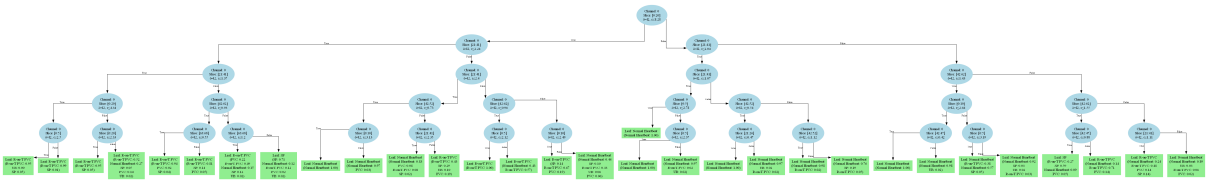
5.2 Univariate + Multi-class Classification (ECG5000)

The **ECG5000** dataset consists of 4,500 training samples and 500 test samples, categorized into five distinct classes: *Normal*, *R-on-T PVC*, *PVC*, *SP*, and *UB*. This scenario evaluates the ability to handle larger datasets and more complex multi-class decision boundaries.

5.2.1 Supervised: Tree Pruning and Complexity

Due to the higher number of samples and classes, the initial Prompt Trees grew significantly in complexity. An unpruned tree is compared against a version optimized via Reduced Error Pruning.

The pruning process reduced the tree from 31 leaves to just 9. The impact on performance was positive: the original tree achieved a test accuracy of 91.60%, while the pruned tree reached **92.00%**. This demonstrates that even in a complex 5-class environment, a smaller Prompt Tree can achieve superior generalization by retaining only the most statistically significant reference slice tests.



(a) Original Tree (Highly Complex)

(b) Pruned Tree (Optimized)

Figure 4: Prompt Tree structure for ECG5000. Pruning drastically reduced the number of nodes while maintaining high class-distribution purity at the leaves.

5.2.2 Unsupervised: Multi-class Clustering Analysis

An Isolation Forest comprising 100 estimators was trained on the full ECG5000 training set (4,500 samples). We generated pairwise distance matrices using the three metrics and performed hierarchical clustering to partition the data into 5 target clusters.

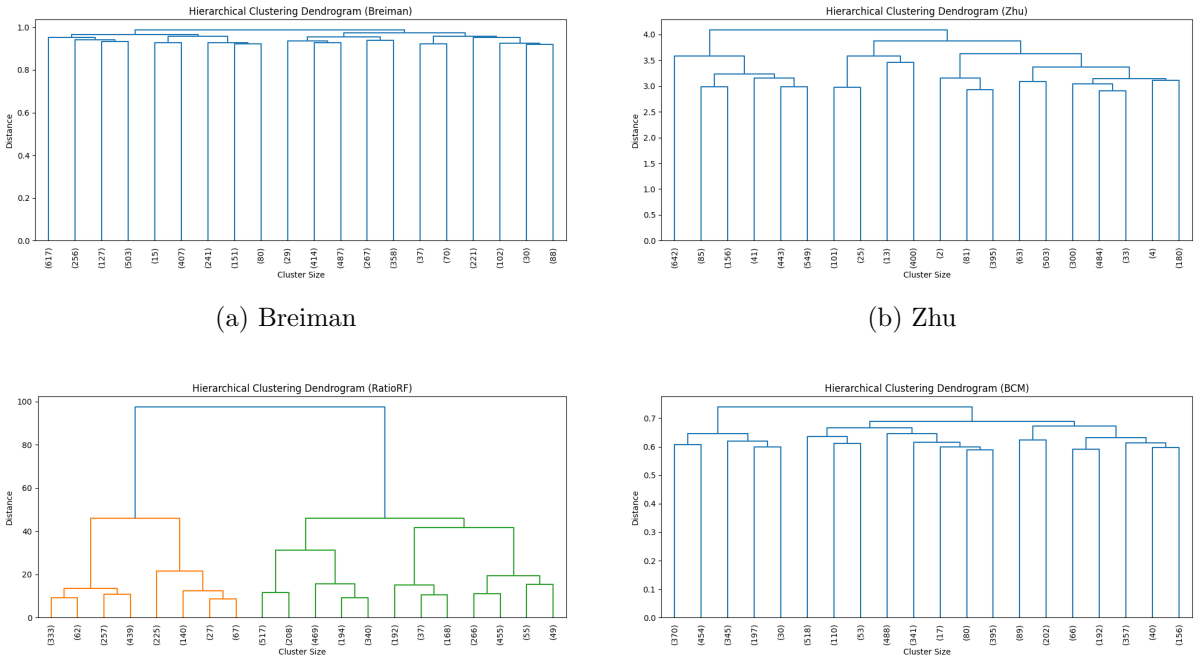
Metric	ARI	Purity	Entropy	Intra-Dist	Inter-Dist
Breiman	0.3821	0.8771	**0.5750**	0.9228	0.9805
Zhu	0.3284	**0.8773**	0.5911	2.9299	3.9555
RatioRF	**0.4049**	0.8253	0.6870	16.0325	77.0429
BCM	0.3756	0.8684	0.5908	0.6029	0.7138

Table 3: ECG5000 Unsupervised Clustering Performance

In the multi-class experiment on the ECG5000 dataset, the distance metrics revealed a

clear trade-off between structural alignment and cluster purity. The leaf-based RatioRF metric achieved the highest structural agreement with the ground truth labels, but at the cost of the lowest purity and highest entropy. Conversely, the Breiman metric provided the strongest overall compromise. It outperformed the path-based BCM metric across all clustering quality indicators, delivering the highest purity and lowest entropy while maintaining a highly competitive Adjusted Rand Index.

Despite its strong clustering performance, Breiman exhibited less ideal geometric properties. While RatioRF produced wide, distinct separation between clusters, Breiman's distances remained highly compressed within a narrow numerical band. This compression does not hinder rank-based approaches like hierarchical clustering, but it offers poor distinction for downstream algorithms relying on absolute distance thresholds.



(a) Breiman (b) Zhu

(c) RatioRF (d) BCM

Figure 5: Hierarchical clustering dendrograms for ECG5000.

5.2.3 Conformal Prediction: Scalability and Calibration

We performed a scalability test for the Conformal Classifier by varying the number of estimators (N) while keeping the significance level $\epsilon = 0.1$.

Estimators (N)	Error Rate	Calibration	Avg Set Size	Point Accuracy
5	0.072	Valid	1.000	92.80%
15	0.058	Valid	1.000	94.20%
20	0.052	Valid	1.000	94.80%
50	**0.042**	**Valid**	**1.000**	**95.80%**

Table 4: Conformal Evaluation for ECG5000 ($\epsilon = 0.1$)

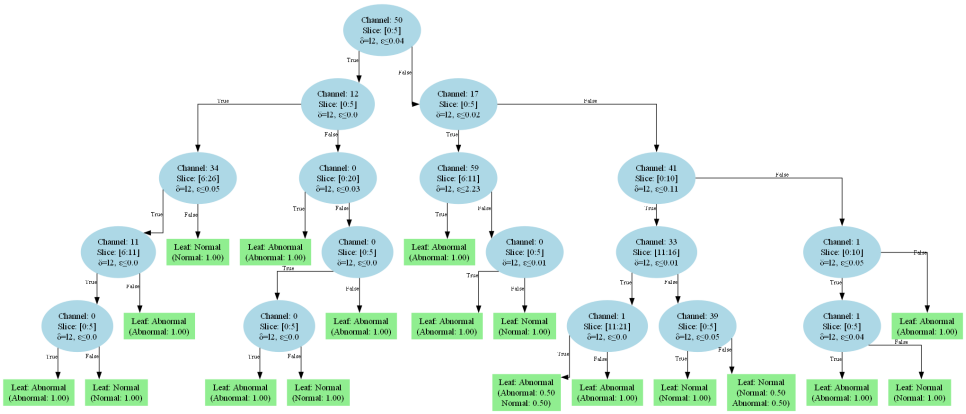
As the forest size increased, the point accuracy improved significantly. In all test cases, the empirical error rate remained below the 0.1 threshold, confirming the classifier is perfectly calibrated. The efficiency was ideal, with an average prediction set size of exactly 1.0, even with as few as 5 estimators.

5.3 Multivariate + Binary Classification (Heartbeat)

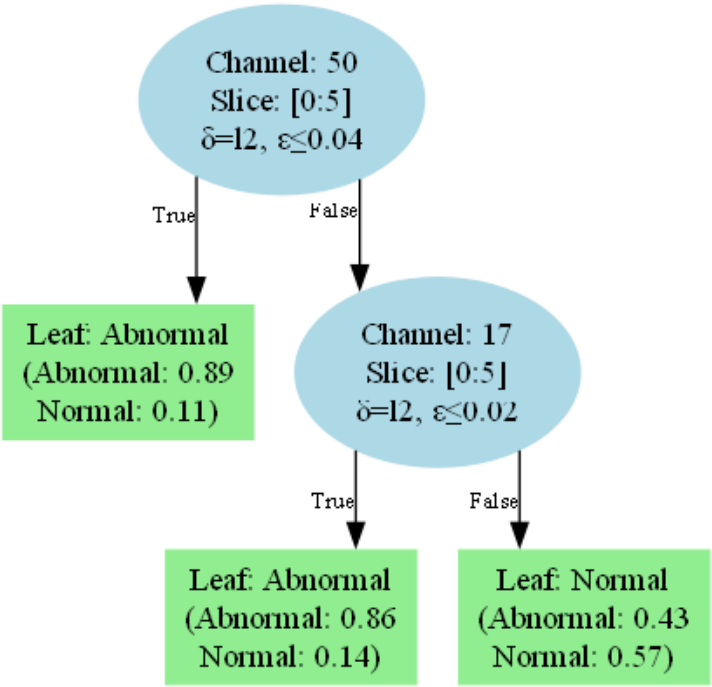
The **Heartbeat** dataset was selected to evaluate the performances on multivariate binary classification, distinguishing between *Normal* and *Abnormal* cardiac audio signals. This dataset contains 204 training and 205 test samples, with each sample consisting of 61 channels and 405 timesteps.

5.3.1 Supervised: Tree Pruning and Generalization

A single `PromptTree` was trained. To improve generalization, **Reduced Error Pruning (REP)** was applied using the validation set. The pruning process significantly compress the model, reducing the total number of leaves from 18 to just 3.
Final evaluation on the test set showed that both the original and pruned trees achieved a test accuracy of **58.05%**. While pruning did not yield a numerical increase in accuracy for this specific split, it produced a much simpler and more interpretable model that captures the core diagnostic signatures of the audio signals with significantly fewer decision paths.



(a) Original Tree



(b) Pruned Tree

Figure 6: Prompt Tree structure for Heartbeat. Pruning successfully consolidated complex branches, moving from 18 leaves to a highly simplified 3-leaf structure while maintaining identical predictive performance.

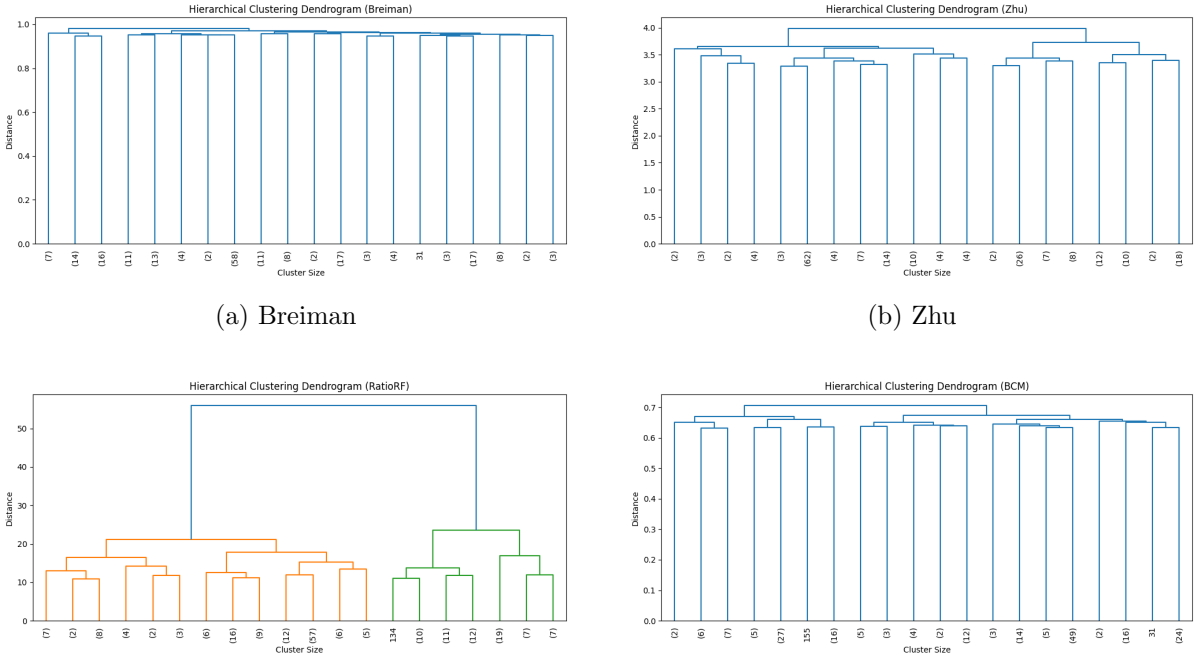
5.3.2 Unsupervised: Clustering and Distance Comparison

An Isolation Forest of 100 trees was utilized to generate distance matrices for the Heartbeat dataset. We evaluated the structural separation achieved by the three distance metrics—Breiman, Zhu, and RatioRF—using hierarchical clustering with average linkage.

Metric	ARI	Purity	Entropy	Intra-Dist	Inter-Dist
Breiman	0.0625	0.7206	0.8427	0.9592	0.9816
Zhu	0.0780	0.7206	**0.8101**	3.4385	3.9885
RatioRF	**0.1008**	0.7206	0.8148	16.2597	56.0010
BCM	0.0766	0.7206	0.8292	0.6506	0.7062

Table 5: Heartbeat Unsupervised Clustering Performance

In the multivariate Heartbeat dataset experiment, all distance metrics struggled heavily to capture the true class structure. While the leaf-based RatioRF technically achieved the highest Adjusted Rand Index (0.1008), an ARI this close to zero indicates that none of the methods effectively separated the data into meaningful ground-truth clusters. All four metrics flatlined at the exact same purity score (0.7206), suggesting the forest essentially grouped the majority class together while failing to isolate the minority patterns.
Despite the poor structural alignment across the board, the geometric behaviors remained consistent with previous tests. RatioRF massively inflated the distances between clusters, while Breiman squashed them into an unusable, narrow band. BCM again offered the most stable distance scaling, though the near-random ARI scores highlight the limitations of this unsupervised forest approach on complex, multivariate signals.



(a) Breiman

(b) Zhu

(c) RatioRF (Leaf-based)

(d) BCM

Figure 7: Hierarchical clustering dendrograms for Heartbeat dataset.

5.3.3 Conformal Prediction: Uncertainty Quantification

A conformal classifier was implemented using a Random Forest with 20 estimators. the model was calibrated using a 30% split of the training data.

Estimators (N)	Error Rate	Calibration	Avg Set Size	Point Accuracy
20	0.063	Valid	1.639	70.73%

Table 6: Conformal Evaluation for Heartbeat ($\epsilon = 0.1$)

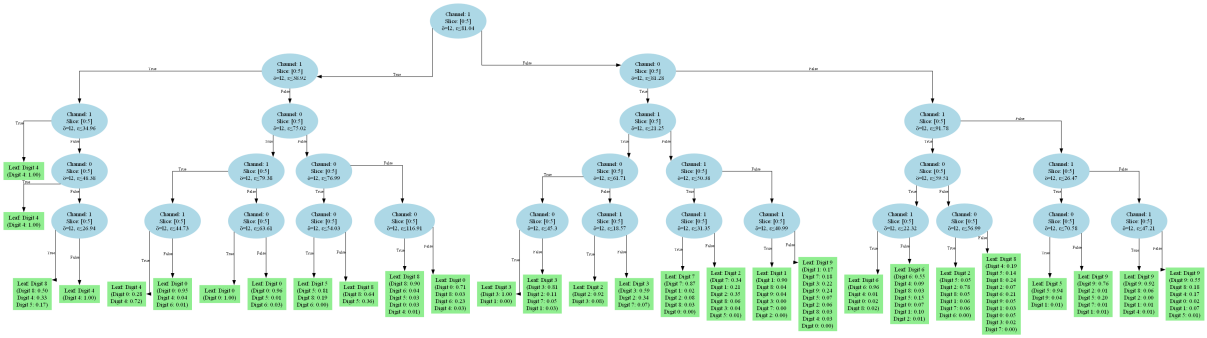
At a significance level of $\epsilon = 0.1$, the classifier achieved an empirical error rate of 0.063 (13 errors out of 205 test samples), confirming it is adequately calibrated as the error rate is below ϵ. The **Efficiency** of the predictor resulted in an average set size of 1.639. The set size greater than 1.0 reflects the model's uncertainty in distinguishing between normal and abnormal signals at the 90% confidence level, though it remains valid.

5.4 Multivariate + Multi-class Classification (PenDigits)

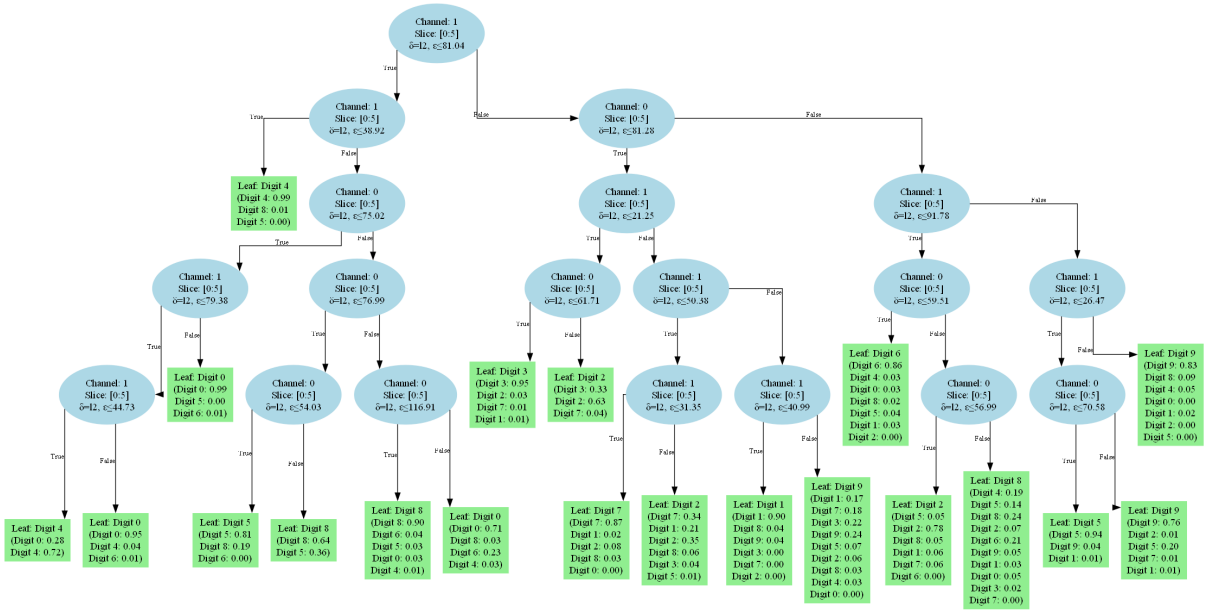
The **PenDigits** dataset was utilized to evaluate the performances in a multivariate multi-class environment. This dataset represents handwritten digits (0-9) as time series of (x, y) coordinates. It contains 7,494 training samples and 3,498 test samples, each featuring 2 channels and 8 timesteps.

5.4.1 Supervised: Tree Pruning and Complexity

Due to the presence of 10 distinct classes and multiple channels, the initial Prompt Trees exhibited high structural complexity. The unpruned tree initially reached a total of 28 leaf nodes. Through the recursive pruning process, the model was significantly optimized, resulting in a more compact and generalized tree with 20 leaf nodes.
The evaluation on the test set confirmed that this structural simplification was beneficial. The original tree achieved a test accuracy of 66.90%, while the pruned tree reached 66.95%. Although the numerical gain in accuracy was marginal, the successful reduction of nearly 30% of the leaf nodes indicates that the pruning process effectively removed redundant decision paths while slightly improving the model's ability to generalize to new handwritten samples.



(a) Original Tree (28 Leaves)



(b) Pruned Tree (20 Leaves)

Figure 8: Prompt Tree structure for PenDigits. Pruning optimized the high-dimensional decision space by consolidating branches, resulting in a more robust model for digit classification.

5.4.2 Unsupervised: Clustering and Distance Comparison

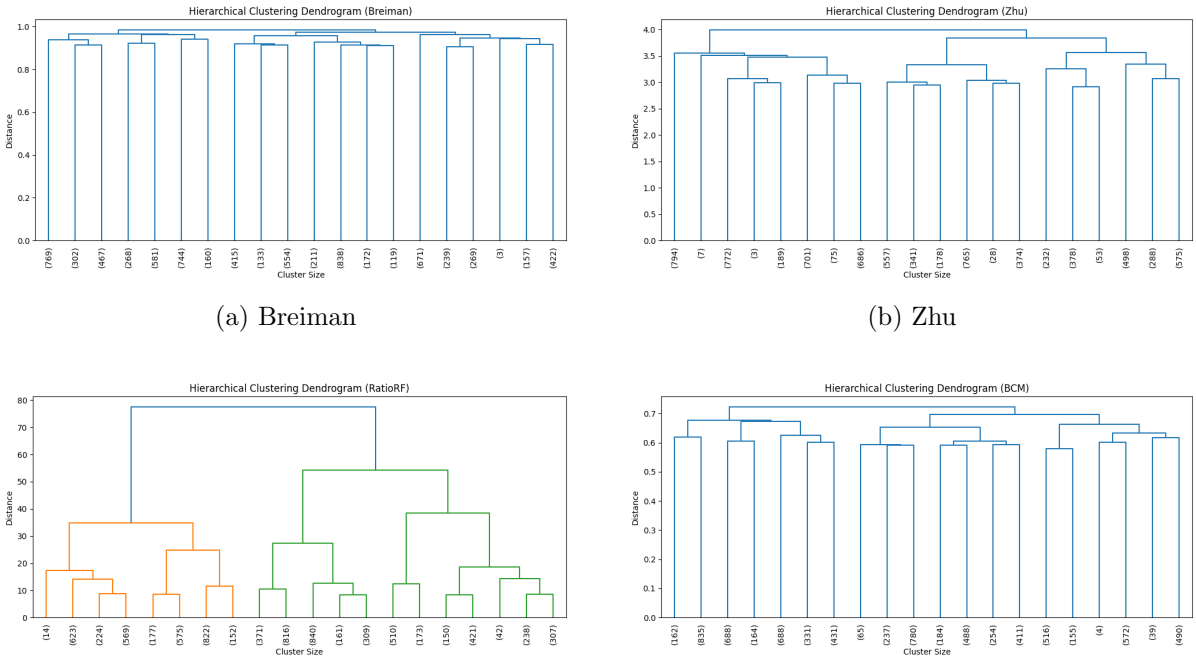
An Isolation Forest with 100 estimators was trained to generate distance matrices for clustering the PenDigits data into 10 target groups. We compared the performance of Breiman, Zhu, and RatioRF metrics.

Metric	ARI	Purity	Entropy	Intra-Dist	Inter-Dist
Breiman	0.4887	0.5990	1.2810	0.8799	0.9765
Zhu	0.4613	0.5799	1.3588	2.6346	3.8313
RatioRF	0.4701	0.6002	1.3640	7.8150	61.6583
BCM	**0.5596**	**0.7048**	**1.0958**	0.5466	0.7011

Table 7: PenDigits Unsupervised Clustering Performance

In the 10-class PenDigits experiment, the path-based BCM metric really shines, easily outperforming all the baseline methods. While Breiman, Zhu, and the leaf-based RatioRF struggled to cleanly untangle the ten different digits—resulting in noticeably messier and more fragmented clusters—BCM delivered the best structural alignment, the highest overall purity, and the low-

est entropy across the board. This clearly highlights how evaluating the full decision paths gives BCM a massive advantage when dealing with complex, multi-class data.
Geometrically, the story remains exactly the same as in previous scenarios. The leaf-based RatioRF artificially inflates the space between clusters, while Breiman squashes all its distances into a tiny numerical window. Once again, BCM hits the sweet spot, providing a stable, well-scaled distance space without the extreme compression or inflation seen in the alternatives.



(a) Breiman

(b) Zhu

(c) RatioRF (Leaf-based)

(d) BCM

Figure 9: Hierarchical clustering dendrograms for PenDigits dataset.

5.4.3 Conformal Prediction: Uncertainty Quantification

A conformal classifier was implemented using 20 estimators and a 30% calibration split. Despite the complexity of the 10-class problem, the model demonstrated excellent reliability.

Estimators (N)	**Error Rate**	**Calibration**	**Avg Set Size**	**Point Accuracy**
20	0.086	Valid (Error $\leq \epsilon$)	1.000	91.42%

Table 8: Conformal Evaluation for PenDigits ($\epsilon = 0.1$)

The classifier proved to be adequately calibrated with an empirical error rate of 8.6%, which is below the 10% significance threshold. Notably, the **Efficiency** remained perfect with an average set size of 1.0, indicating that the multivariate Prompt Forest can maintain high confidence even when distinguishing between a large number of handwriting classes.

6 Conclusion

This implementation provides a robust and flexible framework for time series analysis. By combining the interpretability of Prompt Trees with the power of Random Forests and the statistical guarantees of Conformal Prediction, the system is well-suited for diverse biomedical classification tasks.